SpendNG; Product Requirements Document

Section 1: Product Overview

1.1 Product Name

SpendNG

A working title used across all design, development, and planning assets throughout the product lifecycle. The name signals its core purpose; spending awareness; and its national context: Nigeria.

1.2 Product Vision

To help Nigerian earners automatically capture, organize, and retrieve their complete financial activity across every bank and fintech app they use; without ever requesting a bank statement.

1.3 Problem Statement

Nigerian earners; whether salaried employees, freelancers, or small business owners; routinely transact across three to five financial platforms simultaneously. A typical user might receive salary into a GTBank account, collect client payments via Opay, pay bills through Kuda, and send transfers on Moniepoint. Each platform sends transaction alerts independently, either by SMS or email, but none of them talk to each other.

When financial accountability becomes necessary; for tax filing with FIRS, producing expense records for a client, verifying income for a loan application, or simply understanding where money went; users have no unified record to rely on. Their only options are to manually request statements from each institution (a slow, sometimes costly process), scroll through hundreds of SMS alerts, or attempt to reconstruct history from memory.

This problem is not a matter of financial irresponsibility. It is a structural gap: the tools that exist either require bank API access that most Nigerian fintechs do not offer, target a foreign market entirely, or demand so much manual input that they are abandoned within days.

The result is that most Nigerian earners operate financially blind; spending without records, filing taxes by guesswork, and unable to produce clean financial history when it matters most.

1.4 Product Thesis

SpendNG should exist now for three reasons.

First, the data is already there. Every bank and fintech app in Nigeria sends transaction alerts via SMS or email the moment money moves. These alerts contain the exact information needed to build a financial record; amount, merchant, date, account, and direction of flow; and they arrive in real time without any bank partnership required.

Second, the infrastructure is ready. Android and iOS both provide APIs that allow apps to read incoming SMS messages and connect to email providers via OAuth. A parsing layer built on top of these standard channels can reconstruct a user's full financial history from day one of installation, and continue building it passively from that point forward.

Third, regulatory pressure is increasing. FIRS is expanding its digital tax infrastructure and the expectation that Nigerians; particularly self-employed workers and business owners; will file accurate income and expense records is growing. SpendNG gives users the records they will eventually need, built automatically from the alerts they are already receiving.

Section 2: Market Research

2.1 Nigerian Financial Behavior

Understanding how Nigerian earners actually manage money is foundational to building a product that fits their lives rather than fighting against their habits.

Fintech Adoption

Nigeria has one of the fastest-growing fintech ecosystems in Africa. As of 2024, fintechs like Opay, Kuda, Moniepoint, PalmPay, and Chipper Cash collectively serve tens of millions of active users. Adoption is not driven by preference for innovation alone; it is driven by practical necessity. Traditional banks impose queues, transaction charges, and friction that fintechs have eliminated. For many users, a fintech wallet is now the primary account for daily transactions, while a legacy bank account remains open for salary receipt or formal savings.

Multi-Bank and Multi-Wallet Usage

The average financially active Nigerian under 40 operates across at least two to four financial platforms simultaneously. A common pattern looks like this: salary received into a First Bank or

Access Bank account, airtime and bills paid through Opay or PalmPay for the zero-fee transfers, savings held in Kuda or PiggyVest, and business payments received via Moniepoint or a dedicated GTBank account. This fragmentation is not confusion; it is rational optimization. Each platform offers something slightly different, and users have learned to exploit the differences. The side effect is that no single platform ever holds the complete financial picture.

Cash and Transfer Behavior

Despite fintech growth, cash remains deeply embedded in Nigerian daily life. Market purchases, transport, and informal services are predominantly cash-based. This means a meaningful portion of every user's spending will always be invisible to any alert-based tracking system. SpendNG must acknowledge this honestly in its UX rather than pretend it captures everything. What it can capture; digital transfers, card payments, and bank-to-bank transactions; still represents the majority of structured, traceable spending for its target users.

Salary and Side Hustle Patterns

The rigid salary-only income model does not reflect Nigerian economic reality. A large proportion of formally employed Nigerians also run side businesses, take freelance contracts, or receive informal income from family or community. Freelancers may receive payments in naira from local clients and in USD, GBP, or USDC from international clients via Grey, Geegpay, or Lemfi. This income diversity means SpendNG must handle multiple transaction types, currencies, and alert formats without assuming a clean monthly salary structure.

2.2 Competitor Analysis

Mono

Mono is a financial data infrastructure company that provides bank account connectivity via APIs for businesses. It allows developers to pull transaction history, account balances, and identity data directly from Nigerian banks with user consent. Mono does this well; it is reliable, growing its bank coverage, and trusted by developers building fintech products. Where it fails everyday users is that it is not a consumer product. It has no interface for personal finance management, no transaction categorization layer for individuals, and no mechanism for capturing fintech wallet transactions from platforms that have not integrated its API. For SpendNG's target user, Mono is invisible infrastructure, not a solution.

Okra (now deprecated for consumers)

Okra operated a similar open banking API model to Mono and at one point explored consumer-facing features. It has since pivoted away from the individual user market. Its brief consumer experiment demonstrated both the appetite for unified financial data and the difficulty of solving it purely through bank APIs in a market where API coverage remains incomplete.

YNAB (You Need A Budget)

YNAB is a US-based budgeting application built around zero-based budgeting principles. It is genuinely excellent at what it does; helping disciplined users allocate every naira before spending it. The problems for Nigerian users are numerous. It requires manual transaction entry or US/UK bank integrations that do not exist for Nigerian institutions. Its pricing is in USD. Its mental model assumes a predictable income structure that does not match Nigerian financial reality. The few Nigerians who use it do so despite the product, not because of it.

Mint (discontinued 2024)

Mint was a US personal finance app that aggregated bank accounts and auto-categorized spending. It was the closest global equivalent to what SpendNG aims to build. It was discontinued in early 2024, partly because its aggregation model was too dependent on US banking infrastructure. Its core insight; that users want automatic, effortless financial tracking; remains valid and entirely unserved in Nigeria.

Kuda

Kuda is a Nigerian neobank, not a financial tracking product, but it deserves mention because many users treat its in-app transaction history as a de facto spending tracker. Kuda does this well for transactions that happen within Kuda. The limitation is obvious: it only sees what moves through its own platform. A user who spends across Kuda, GTBank, and Opay gets a partial picture at best. Kuda also has no tax export, no categorization engine, and no cross-platform aggregation.

2.3 Market Gap

No product currently available to Nigerian consumers automatically builds a unified, categorized, and exportable financial record from transaction alerts across multiple banks and fintech apps; without requiring bank API access, manual data entry, or a foreign bank account.

SpendNG occupies this gap entirely. It works with the infrastructure already in every Nigerian's pocket; SMS alerts and email notifications; and converts them passively into a financial record that is searchable, categorized, and ready for tax filing, expense reporting, or personal review. It requires no bank partnerships, no API negotiations, and no behavior change from the user. The data is already arriving. SpendNG simply catches it.

Section 3: User Research

3.1 Persona Framework

SpendNG serves three equally prioritized user groups in version 1. Each persona below is built from the behavioral realities of Nigerian earners rather than generic demographic assumptions. Names are representative archetypes, not fictional characters.

Persona 1: The Salary Earner

Representative Profile Tunde, 29. Works as a mid-level operations analyst at a logistics company in Lagos. Takes home ₦280,000 monthly. Lives in Yaba, commutes daily, and splits financial activity across three platforms without thinking much about it.

Financial Behavior Tunde receives his salary into a First Bank account on the 27th of every month. He immediately transfers a portion to Kuda for daily spending because Kuda's zero-fee transfers make it easier for daily use. He pays rent annually via bank transfer from First Bank, buys food and transport with cash, and uses his GTBank debit card for online purchases and subscriptions. He has a PiggyVest flex savings account he tops up irregularly. By the end of the month, he genuinely cannot account for where a significant portion of his money went. He knows roughly what he earns. He has no idea what he actually spends.

Goals

- Understand his monthly spending without having to request statements or scroll through SMS alerts manually.
- Know how much he spends on food, transport, and discretionary items each month.
- Have something to show an employer or financial institution if income verification is ever required.
- File any future tax obligations without scrambling to reconstruct records.

Frustrations

- His bank app shows him First Bank transactions only. Kuda shows Kuda only. Nothing shows him everything together.
- Requesting a formal bank statement from First Bank takes time and sometimes involves a branch visit.
- He has tried budgeting apps before but abandoned them within two weeks because manual entry felt like a second job.
- He does not trust himself to maintain any system that requires ongoing effort.

Devices Primary device: Android (Tecno Camon). Secondary: laptop used occasionally for work, rarely for personal finance.

Technical Comfort Moderate. Comfortable installing apps, granting permissions, and navigating standard mobile interfaces. Will not read documentation. Abandons any flow with more than three steps if the value is not immediately obvious.

Persona 2: The Freelancer

Representative Profile Amara, 26. A freelance UX designer based in Abuja. She works with local startups and international clients simultaneously. Her income is irregular, multi-currency, and spread across more platforms than she can comfortably track.

Financial Behavior Amara receives naira payments from Nigerian clients directly into her Moniepoint business account or via bank transfer to her Access Bank account. International clients pay in USD through Grey or Geegpay, which she converts to naira at favorable rates before spending. She tracks expenses loosely in a Notes app on her phone and maintains a rough mental model of what she has earned each month. At tax time; or when a client requests an invoice history; she spends several hours reconstructing records she should have been maintaining all year. She also has legitimate business expenses (software subscriptions, data, equipment) that she knows she could deduct if she had clean records.

Goals

- Have an automatic record of every payment received across all platforms so invoicing and income verification are effortless.
- Categorize business expenses separately from personal spending.
- Export a clean financial summary at the end of the year for tax purposes or client reporting.
- Understand her effective monthly income across naira and converted foreign currency earnings.

Frustrations

- No single app understands that she earns in multiple currencies from multiple sources.
- She cannot easily separate what she spends for work from what she spends personally when everything lives in the same account.
- She has missed deductible expenses in past tax filings simply because she had no record of them.
- International payment platforms like Grey send email confirmations but no Nigerian finance app reads those emails automatically.

Devices Primary device: iPhone 13. Secondary: MacBook Pro used daily for client work.

Technical Comfort High. Comfortable with OAuth flows, app permissions, and connecting services. Will explore settings. Will read an explanation if it is clearly written. Has strong opinions about design quality and will abandon poorly designed products regardless of utility.

Persona 3: The Small Business Owner

Representative Profile Chidi, 34. Runs a small tailoring and fashion retail business in Enugu with two part-time staff. He sells in-person and takes orders via WhatsApp. Payments come in through bank transfers, POS, and occasionally cash. He has a business account but also uses his personal account when customers send to the wrong number.

Financial Behavior Chidi receives business payments across his Zenith Bank business account, a personal UBA account, and an Opay account he opened because several customers preferred it. He pays suppliers in cash and via transfer. He has no bookkeeper and no accounting software. At the end of each month he tries to reconcile income by reviewing his bank alerts, but frequently misses transactions that came through on busy days. His accountant, who helps him file annual taxes, has told him repeatedly that his records are too disorganized to maximize deductions. He knows this is costing him money but has no clean solution.

Goals

- Automatically capture every business payment received across all accounts without manual entry.
- Separate business income and expenses from personal spending over time.
- Give his accountant something usable at year-end; a structured export rather than a folder of screenshots.
- Understand which months are strong and which are slow so he can plan supplier purchases better.

Frustrations

- Business and personal finances are mixed because that is how customers pay, and separating them retroactively is painful.
- POS transactions do not always generate SMS alerts immediately, creating gaps in his records.
- He has no time during the business day to maintain any system that requires active input.
- Every accounting tool he has seen is either too expensive, too complex, or built for a market that is not Nigeria.

Devices Primary device: Android (Samsung Galaxy A-series). Also uses a shared tablet at the shop for POS management.

Technical Comfort Low to moderate. Comfortable with WhatsApp, mobile banking apps, and basic Android navigation. Distrustful of apps that ask for too many permissions without clear explanation. Needs onboarding to feel trustworthy and simple before he will engage with deeper features.

3.4 Cross-Persona Observations

Three patterns emerge consistently across all three personas that should directly shape SpendNG's design decisions.

The first is **passive capture as a non-negotiable**. All three users have tried and abandoned active tracking systems. Any feature that requires ongoing manual input will be used for days and then ignored. SpendNG must do the work automatically or it will not be used at all.

The second is **trust through transparency**. All three users will encounter a permission request; SMS access, email OAuth; that could feel invasive if not carefully explained. The value exchange must be made explicit and the language must be plain. Chidi in particular will uninstall before granting permissions he does not understand.

The third is **export as the proof of value**. For Tunde, export is a future safety net. For Amara, it is a quarterly necessity. For Chidi, it is the primary reason to use the product at all. Export must be present in version 1 and it must produce something that looks credible and clean; not a raw data dump.

Section 4: Jobs To Be Done

4.1 Framework Note

Jobs To Be Done are written from the user's perspective, in the moment of need. Each job follows the structure: **When** [situation] **I want to** [motivation] **so I can** [expected outcome]. They are organized by trigger type; recurring needs, event-driven needs, and anxiety-driven needs; because the timing of a job shapes how SpendNG must deliver on it.

4.2 Recurring Jobs

These are jobs users need done on a regular basis; daily, weekly, or monthly; as part of normal financial life.

Job 1 When I receive or spend money through any of my bank or fintech accounts, I want the transaction recorded automatically so I can build a financial history without any effort on my part.

This is the foundational job. Everything SpendNG does depends on solving this one reliably. If passive capture fails, no other job can be completed.

Job 2 When I check my phone at the end of the day, I want to see a clear summary of what moved through my accounts so I can stay aware of my financial activity without opening five different banking apps.

This job drives the home screen and notification design. The user is not asking for analysis; they are asking for a single, consolidated view of what already happened.

Job 3 When a new month begins, I want to see what I spent last month broken down by category so I can understand my spending patterns without manually reviewing individual alerts.

This job drives the categorization engine and the monthly summary view. Tunde and Amara both need this regularly. It does not require budgeting features; it requires accurate categorization and a readable summary.

Job 4 When I receive a payment from a client or employer, I want it logged with the correct amount, source, and date so I have a reliable income record I did not have to create myself.

This job is especially critical for Amara the freelancer, who needs income records that are credible and timestamped, not reconstructed from memory weeks later.

4.3 Event-Driven Jobs

These are jobs triggered by specific life or business events; tax season, loan applications, client requests; that do not happen every day but carry high stakes when they do.

Job 5 When tax season arrives, I want to retrieve a categorized record of my income and expenses for the full year so I can file accurately without requesting statements from multiple banks or guessing at figures.

This is the highest-stakes job in the product. For Amara and Chidi, this job alone justifies installing and maintaining SpendNG all year. The export feature exists entirely to serve this moment.

Job 6 When a client, employer, or financial institution asks me to verify my income or spending history, I want to produce a clean, exportable document so I can respond quickly and credibly without embarrassment or delay.

This job covers loan applications, tenancy verification, client due diligence, and any other scenario where financial credibility must be demonstrated externally. The output must look professional; not like a screenshot.

Job 7 When I need to give my accountant my annual financial records, I want to export a structured file that covers all my accounts so I can hand over something organized rather than a folder of bank alert screenshots.

This is Chidi's primary job. It also applies to Amara. The export format must be structured enough that an accountant can work with it directly; a CSV or PDF with clear columns and categories, not a raw data export.

Job 8 When I am making a large financial decision; paying rent, buying equipment, hiring help; I want to review my recent income and spending so I can make the decision with actual numbers rather than a rough mental estimate.

This job drives the search and summary features. The user is not running a formal budget; they are doing a quick gut-check with real data before committing to something significant.

4.4 Anxiety-Driven Jobs

These are jobs triggered by financial uncertainty, doubt, or the quiet stress of not knowing where money went. They are less structured than event-driven jobs but occur frequently and represent some of the deepest product value.

Job 9 When I feel like I am spending too much but cannot identify where, I want to search my transaction history by category or merchant so I can find the pattern that is draining my account without scrolling through hundreds of alerts manually.

This job drives the search and filter experience. The user knows something is wrong but cannot name it. SpendNG's job is to make the answer findable in under thirty seconds.

Job 10 When I suspect a transaction on my account is wrong or unfamiliar, I want to find that specific transaction quickly so I can confirm whether it is legitimate or flag it for investigation.

This job reinforces the need for fast, flexible search. It also suggests that SpendNG's transaction detail view must show enough information; source, amount, date, raw alert text; to help a user make a judgment call.

Job 11 When I have not checked my finances in a while, I want to open the app and immediately understand what has happened across all my accounts so I do not have to piece together the picture from multiple sources.

This job defines the quality bar for the home screen. A user returning after two weeks of neglect should feel oriented within ten seconds of opening the app; not overwhelmed.

Job 12 When I am about to end the month with less money than expected, I want to understand what changed compared to last month so I can identify whether this is a spending problem, an income problem, or a one-off event.

This job is the closest SpendNG version 1 gets to financial insight without crossing into budgeting territory. It requires month-over-month comparison data and enough categorization accuracy to make the comparison meaningful.

4.5 Jobs SpendNG Explicitly Does Not Do in Version 1

Being clear about what SpendNG is not hired to do is as important as defining what it is. In version 1, SpendNG is not hired to:

- Tell users what to spend in the future. That is budgeting. It is out of scope.
- Manage investments or savings goals. That is wealth management. It is out of scope.
- Send money or initiate transactions. That is a neobank. It is out of scope.
- File taxes on the user's behalf. That is tax software. SpendNG produces the records; the filing happens elsewhere.
- Replace an accountant. SpendNG gives Chidi something to hand his accountant; it does not replace the accountant's judgment.

These boundaries protect version 1 from scope creep and keep the product focused on the single job it does better than anything else available in Nigeria: **building an automatic, unified financial record from the alerts already arriving on a user's phone.**

Section 5: Product Scope

5.1 In Scope; Version 1

These are the features that must ship in version 1. Each one is essential to delivering the core value proposition. None of them are optional.

SMS Ingestion SpendNG shall read incoming and existing SMS messages on the user's device, identify transaction alerts from Nigerian banks and fintech apps, and parse them into structured transaction records. This is the primary data ingestion channel for both Android and iOS users and the backbone of the entire product. Without it, SpendNG has no data to work with.

Email Ingestion SpendNG shall connect to Gmail and Outlook accounts via OAuth and process transaction alert emails in the background. This channel captures alerts from platforms that deliver primarily by email; including international payment services like Grey, Geegpay, and Lemfi; and serves as the primary ingestion channel for both Android and iOS users where SMS reading is restricted by the operating system.

Transaction Parsing SpendNG shall extract the core fields from every identified alert: transaction amount, currency, sender or merchant name, transaction type (debit or credit), transaction date and time, account or platform of origin, and available balance where included. Parsing must handle the wide variety of alert formats used by Nigerian banks and fintechs, which differ significantly in structure, language, and punctuation.

Auto-Categorization SpendNG shall automatically assign every parsed transaction to a predefined financial category; such as Food, Transport, Utilities, Airtime, Transfers, Business Income, or Salary; based on merchant name matching, keyword rules, and transaction type. Categorization must apply at the moment of parsing and must be editable by the user after the fact.

Manual Transaction Entry SpendNG shall allow users to manually add transactions that did not generate an alert; primarily cash transactions; by entering amount, category, date, description, and direction. This feature acknowledges that cash spending is a significant portion of Nigerian financial activity and gives users a way to build a complete picture even where automatic capture is impossible.

Transaction Search SpendNG shall provide a fast, flexible search interface that allows users to find transactions by merchant name, amount, category, date range, account, and free-text keyword. Search results must return within two seconds and must be filterable and sortable by multiple parameters simultaneously.

Dashboard and Summaries SpendNG shall present a home screen dashboard showing total income and spending for the current month, a categorical breakdown of spending, recent transaction activity, and a month-over-month comparison. The dashboard must be readable at a glance without requiring the user to navigate into sub-screens to understand their current financial position.

Export SpendNG shall allow users to export their transaction history as a PDF report or CSV file, filtered by date range, category, or account. The PDF export must be formatted cleanly enough to present to an accountant, employer, or financial institution. The CSV export must be structured with clearly labeled columns suitable for import into spreadsheet or accounting software.

Onboarding Flow SpendNG shall guide new users through account creation, permission granting, and initial data import in a flow that takes no longer than three minutes to complete and clearly explains the value of each permission before requesting it. The onboarding flow must handle partial permission grants gracefully and must not block users from accessing the product if they decline optional permissions.

User Account and Authentication SpendNG shall support account creation via email and password, with phone number as an optional identifier. Authentication must include a PIN or biometric lock for app access. User data must be tied to an account so that it persists if the app is reinstalled or the user switches devices.

5.2 Out of Scope; Version 1

These features are intentionally excluded from version 1. Each exclusion is a deliberate product decision, not an oversight.

Budgeting and Spending Limits Setting forward-looking spending limits, envelope budgets, or savings targets is excluded from version 1. Budgeting requires a behavioral commitment from users that is distinct from passive tracking. Introducing it in version 1 would complicate the UX, lengthen onboarding, and distract from the core value of automatic record-building. It will be considered for version 2 once retention data confirms users are returning to review their records regularly.

Investment and Savings Tracking Monitoring investment portfolios, PiggyVest savings targets, or stock holdings is out of scope. These require entirely different data models, integrations, and user mental models than transaction tracking. Mixing them into version 1 would dilute the product's focus.

Bank API Integration Connecting directly to bank APIs via Mono, Okra, or any open banking infrastructure is out of scope for version 1. API coverage across Nigerian fintechs remains incomplete, the integration cost is high, and the SMS plus email approach already covers the majority of use cases without it. API integration will be explored as a supplementary channel in a future version.

Shared or Joint Accounts Features for tracking finances shared between partners, household members, or business co-owners are excluded. Multi-user financial management introduces significant complexity in data ownership, permissions, and conflict resolution that is not appropriate for version 1.

Bill Reminders and Payment Scheduling Proactive notifications for upcoming bills or recurring payment schedules are out of scope. While useful, this feature requires the product to make predictions about future behavior based on historical patterns; a capability that requires more data and validation than version 1 can support.

Accounting Software Integration Direct integration with QuickBooks, Wave, Sage, or any third-party accounting platform is excluded. Version 1 serves Chidi's need through clean CSV and PDF export. Direct integrations require API partnerships and maintenance overhead that are not justified until the user base and use case are validated.

Cryptocurrency Tracking Tracking crypto wallets, USDT holdings, or blockchain transactions is out of scope for version 1. The user base and regulatory environment in Nigeria make this a secondary priority. It will be revisited when the core product is stable.

In-App Financial Advice AI-generated or rule-based financial recommendations are excluded from version 1. SpendNG is a record-keeping tool, not a financial advisor. Introducing advice features without sufficient transaction history or personalization infrastructure would produce generic suggestions that erode trust rather than build it.

Section 6: Functional Requirements

6.1 SMS Ingestion

Purpose To automatically capture transaction alerts delivered via SMS from Nigerian banks and fintech platforms and convert them into structured transaction records.

Inputs

- Incoming SMS messages received after app installation
- Existing SMS messages in the device inbox at the time of first installation
- User-granted SMS read permission (Android only)

Outputs

- Parsed transaction records stored in the transactions table
- Ingestion log entry per processed message stored in the sync_logs table
- User notification confirming new transaction captured (optional, user-configurable)

Behavior The system shall request SMS read permission during onboarding with a plain-language explanation of why it is needed. Upon permission grant, the system shall scan the existing SMS inbox for historical transaction alerts and process them in the background without blocking the UI. From that point forward, the system shall monitor for new incoming SMS messages in real time using a background service. Every SMS received shall be passed through a sender identification filter. Messages from unrecognized senders that do not match any known bank or fintech sender ID shall be discarded without processing. Messages from recognized senders shall be passed to the transaction parser.

Supported Senders; Initial Coverage Version 1 must support SMS alerts from the following institutions at launch: GTBank, Access Bank, First Bank, Zenith Bank, UBA, Fidelity Bank, Sterling Bank, Kuda, Opay, Moniepoint, PalmPay, and Wema Bank. Additional senders will be added based on user feedback during beta.

Edge Cases

- Duplicate SMS alerts for the same transaction (common with some banks): the system shall detect duplicates by comparing amount, date, and account reference and shall store only the first occurrence, logging the duplicate in sync_logs.
- Malformed or partially delivered SMS: the system shall attempt parsing and, if extraction fails for required fields, shall store the raw message in a failed_parses log for review.
- SMS from unsupported banks: the system shall store unrecognized transaction-like messages in a pending review queue that the user can manually parse and submit for pattern training.
- User revokes SMS permission after initial grant: the system shall detect the permission change, notify the user that SMS ingestion is paused, and prompt re-grant without deleting any previously captured data.

Failure States

- Permission denied on first request: user is shown an explanation screen and directed to grant permission manually from device settings. SMS ingestion remains inactive until permission is granted.
- Background service killed by device OS (common on aggressive Android skins like MIUI and ColorOS): the system shall implement a heartbeat check and prompt the user to whitelist SpendNG in battery optimization settings during onboarding.
- Parser returns null for all required fields: the message is logged as a failed parse and excluded from the transactions table until manually reviewed.

6.2 Email Ingestion

Purpose To capture transaction alert emails from banks, fintechs, and international payment platforms and convert them into structured transaction records.

Inputs

- OAuth access token for Gmail, Yahoo, or Outlook granted during onboarding
- Incoming emails from recognized financial senders
- Historical emails in the connected inbox at the time of first connection

Outputs

- Parsed transaction records stored in the transactions table
- Ingestion log entries in sync_logs
- OAuth token refresh logs

Behavior The system shall present Gmail and Outlook connection options during onboarding as optional but strongly recommended steps. Upon OAuth grant, the system shall perform an initial historical scan of the inbox, filtering for emails from recognized financial sender addresses. Historical emails shall be processed in the background in reverse chronological order, with the most recent processed first so that the user sees recent data immediately. From that point forward, the system shall poll for new emails at a configurable interval; defaulting to every fifteen minutes; using the connected account's API. Only emails from recognized financial senders shall be processed. The system shall never access, read, or store email content from non-financial senders under any circumstances.

Supported Email Senders; Initial Coverage Version 1 must support email alerts from: Grey, Geegpay, Lemfi, Paystack (payment confirmations), Flutterwave (payment confirmations), Piggyvest, and Cowrywise, in addition to email alert formats from the banks listed in the SMS section.

Edge Cases

- HTML email with complex formatting: the system shall parse the HTML body and extract structured fields from the rendered text, not the raw HTML markup.
- Forwarded transaction emails: the system shall detect forwarded headers and attempt to parse the original email body, flagging the record as sourced from a forwarded message.
- Email thread with multiple transactions: the system shall parse each transaction independently and create separate records for each one identified in the thread.
- User connects a shared or work email account: the system shall still only process emails from recognized financial senders, ensuring non-financial email content is never accessed.

Failure States

- OAuth token expires: the system shall detect the expired token, notify the user, and prompt reconnection with a single tap.

- Email provider API rate limit reached: the system shall queue unprocessed emails and retry after the rate limit window resets, without notifying the user unless the delay exceeds two hours.
- Email format not recognized: the message is logged in failed_parses and excluded from the transactions table.

6.3 Transaction Parsing

Purpose To extract structured financial data from raw SMS and email alert text and produce normalized transaction records.

Inputs

- Raw SMS message text
- Raw email body text
- Sender identification metadata

Outputs A normalized transaction object containing:

- transaction_id (system generated)
- amount (decimal)
- currency (default NGN, with detection for USD, GBP, EUR)
- transaction_type (credit or debit)
- merchant_or_sender (string)
- account_name (string)
- account_number (last four digits where available)
- transaction_date (timestamp)
- available_balance (decimal, where included in alert)
- raw_source_text (original alert text, stored for audit)
- source_channel (sms or email)
- parse_confidence (high, medium, or low)

Behavior The parser shall maintain a library of regex patterns and extraction templates for each supported bank and fintech sender. Each template shall define the expected structure of that sender's alert format, including field positions, delimiters, currency symbols, and date formats. When a message is received, the system shall identify the sender, select the appropriate template, apply extraction patterns, and validate that required fields; amount, transaction type, and date; have been successfully extracted before creating a transaction record. Parse confidence shall be set to high when all fields are extracted cleanly, medium when required fields are extracted but optional fields are missing, and low when extraction required fallback heuristics.

Edge Cases

- Amount formatting variations: the parser must handle ₦1,000.00, NGN 1000, 1,000 naira, and 1000.00 as equivalent representations.

- Date format variations: the parser must handle DD/MM/YYYY, MM-DD-YYYY, and natural language formats such as "today" or "27 Apr 2025."
- Alerts in Yoruba, Igbo, or Hausa: version 1 shall log these as unparseable and queue them for pattern addition. Full multilingual parsing support is a version 2 feature.
- Truncated SMS: if an SMS is split across multiple messages by the carrier, the system shall attempt to concatenate them before parsing.

Failure States

- Required fields cannot be extracted: transaction is not created. Raw message is stored in failed_parses with sender metadata for pattern improvement.
- Amount extracted as zero: transaction is flagged for user review and not automatically committed to the transactions table.
- Conflicting field values within a single alert: the parser shall take the most specific value; for example, preferring an explicitly labeled amount field over a balance change calculation.

6.4 Auto-Categorization

Purpose To automatically assign every parsed transaction to a meaningful financial category without requiring user input.

Inputs

- Parsed transaction object
- Merchant or sender name
- Transaction type (credit or debit)
- Categorization rules table

Outputs

- Category assignment added to the transaction record
- Confidence score for the assignment
- Flag for low-confidence assignments requiring user review

Predefined Categories; Version 1 Food and Dining, Transport, Utilities and Bills, Airtime and Data, Transfers Sent, Transfers Received, Salary and Wages, Business Income, Shopping, Entertainment, Health, Education, Rent and Housing, and Uncategorized.

Behavior The system shall maintain a rules table mapping merchant keywords, sender names, and transaction patterns to categories. When a transaction is parsed, the system shall apply rules in priority order: exact merchant name match first, partial keyword match second, transaction type and amount pattern third. Transactions that cannot be matched to any rule with sufficient confidence shall be assigned to Uncategorized and flagged for user review. Users shall be able to recategorize any transaction manually. When a user recategorizes a transaction from a merchant that has previously been miscategorized, the system shall offer to apply the

new category to all future transactions from the same merchant. The system shall learn from user corrections and update its local rules accordingly.

Edge Cases

- Same merchant name used across multiple categories: for example, a supermarket that also sells airtime. The system shall use transaction amount range as a secondary signal to distinguish between category types.
- Transfer between user's own accounts: the system shall attempt to detect self-transfers by matching outbound and inbound transactions of the same amount within a short time window and flag them as internal transfers rather than income or expense.
- Salary payments that do not contain the word salary: the system shall use amount regularity, transaction day patterns, and sender consistency to identify probable salary credits over time.

Failure States

- Rules table is empty or corrupted: the system shall fall back to transaction type categorization only; crediting all inflows as Transfers Received and debiting all outflows as Transfers Sent; and flag all records as requiring review.
- User correction creates a circular rule conflict: the system shall surface the conflict to the user and request clarification rather than applying an ambiguous rule silently.

6.5 Manual Transaction Entry

Purpose To allow users to record transactions that did not generate an alert, primarily cash transactions.

Inputs

- Amount (required)
- Transaction type; credit or debit (required)
- Date (required, defaults to today)
- Category (required, selected from predefined list)
- Description (optional, free text)
- Merchant or payer name (optional)

Outputs

- Transaction record stored in the transactions table with source marked as manual
- Record included in all summaries, searches, and exports

Behavior The manual entry form shall be accessible from a persistent floating action button on the home screen and transaction list. All manually entered transactions shall be visually distinguished from automatically captured transactions in the transaction list with a small

indicator. Manual entries shall participate in all categorization, search, and export features identically to automatically captured transactions.

Edge Cases

- User enters a future date: the system shall allow future-dated entries but flag them visually as pending.
- User enters a duplicate manual transaction: the system shall detect probable duplicates by comparing amount, date, and category and warn the user before saving.

Failure States

- Required fields left blank: form shall validate inline and prevent submission until required fields are complete.

6.6 Transaction Search

Purpose To allow users to find specific transactions quickly using multiple search parameters.

Inputs

- Free-text keyword (merchant name, description, notes)
- Amount or amount range
- Transaction type (credit, debit, or both)
- Category (single or multiple)
- Date range
- Source account or platform
- Source channel (SMS, email, or manual)

Outputs

- Filtered list of matching transactions sorted by relevance or date
- Result count
- Option to export filtered results

Behavior Search shall be available from a persistent search bar on the transaction list screen. Results shall update in real time as the user types. All filter parameters shall be combinable; a user must be able to search for all food transactions between two dates above a specific amount in a single query. Search results shall display the merchant name, amount, category, date, and source platform for each record. Tapping a result shall open the full transaction detail view.

Performance Requirement Search results must return within two seconds for a transaction history of up to fifty thousand records.

Edge Cases

- Search query returns zero results: the system shall display an informative empty state explaining which filters are active and offering a one-tap option to clear all filters.
- User searches by merchant name with a typo: the system shall implement fuzzy matching to surface near-matches alongside exact matches.

Failure States

- Search index corrupted: the system shall fall back to sequential scan with a visible performance warning and trigger a background index rebuild.

6.7 Export

Purpose To allow users to produce a clean, portable record of their transaction history for tax filing, accountant submission, income verification, or personal archiving.

Inputs

- Date range (required)
- Category filter (optional)
- Account or platform filter (optional)
- Export format; PDF or CSV (required)

Outputs

- PDF report containing: cover page with user name and date range, summary statistics, categorical breakdown with totals, and full transaction list with merchant, amount, category, date, and source columns.
- CSV file containing one row per transaction with all structured fields as labeled columns.
- Both formats delivered to the device's downloads folder and shareable via the native share sheet.

Behavior The export flow shall be accessible from the main navigation. The user shall select a date range using a calendar picker, optionally apply filters, select a format, and initiate export. Export generation shall happen on-device where possible to avoid sending raw transaction data to a server. The system shall display a progress indicator for exports covering large date ranges. Upon completion, the user shall be presented with options to save to device, share via any app in the native share sheet, or email directly.

Edge Cases

- Export covers date range with no transactions: the system shall warn the user before generating an empty export and offer to adjust the date range.

- PDF generation fails due to memory constraints on low-end devices: the system shall fall back to CSV export and notify the user.
- User attempts to export more than five years of data: the system shall warn about file size and recommend narrowing the date range.

Failure States

- Device storage insufficient for export file: the system shall calculate required storage before generating and warn the user if space is insufficient.
- Export interrupted by app backgrounding: the system shall resume export when the app is foregrounded rather than starting over.

Section 7: User Flows

7.1 First-Time Onboarding Flow

Entry Trigger User downloads SpendNG from the App Store or Play Store and opens it for the first time.

Flow

1. Welcome screen displays product name, tagline, and a single call to action: Get Started.
2. The user taps Get Started and is presented with a brief three-screen value explanation: what SpendNG does, how it works, and what permissions it needs and why. Each screen is one illustration and two sentences.
3. Account creation screen. User enters email address and creates a password. Alternatively, the user signs up with Google. Phone number is optional at this stage.
4. SMS permission request (Android only). Screen explains in plain language: "SpendNG reads your bank SMS alerts to build your transaction history automatically. We never read personal messages." Two options: Allow and Set Up Later. If Allow is tapped, the OS permission dialog appears. If Set Up Later is tapped, the user proceeds without SMS and is reminded once on day two.
5. Email connection screen. Users are presented with Gmail and Outlook connection buttons and a Skip for Now option. Connecting email is explained as especially useful for international payment platforms. OAuth flow is triggered on selection.
6. Import progress screen. If SMS permission was granted, the system begins scanning the inbox immediately. A progress bar indicates historical import status. The user can tap Continue to proceed to the home screen while import runs in the background.
7. Home screen appears. If transactions have already been imported, they are visible immediately. If import is still running, an animated indicator communicates that SpendNG is working.

Success State User reaches the home screen with at least one transaction visible within sixty seconds of completing onboarding.

Failure State User denies all permissions and skips email. The home screen displays an empty state with a prompt to connect at least one data source, with direct links to the SMS and email setup flows.

7.2 Permission Denial Flow

Entry Trigger User taps Deny on the OS SMS permission dialog during onboarding or revokes permission after installation.

Flow

1. System detects permission denial.
2. SpendNG displays a non-blocking explanation screen: "Without SMS access, SpendNG can't automatically capture your bank alerts. You can still connect your email or add transactions manually."
3. User is presented with three options: Open Settings to grant permission, Connect Email Instead, or Continue Without Permissions.
4. If the user selects Open Settings, the device settings app opens directly to SpendNG's permission screen.
5. If the user selects Continue Without Permissions, they proceed to the home screen with an empty state and a persistent but dismissible banner explaining how to enable data sources.

Success State User grants permission from settings and is returned to SpendNG where import begins immediately.

Failure State User continues with no permissions granted. All automated capture features remain inactive. Manual entry is available. The banner remains visible until at least one data source is connected.

7.3 New Transaction Captured Flow

Entry Trigger A new bank SMS alert or transaction email arrives on the user's device.

Flow

1. Background service receives the incoming message.
2. Sender filter identifies the message as a transaction alert.
3. Parser extracts transaction fields and creates a record.

4. Categorization engine assigns a category.
5. If categorization confidence is high, the record is committed to the transactions table silently.
6. If categorization confidence is low, the record is committed but flagged for review.
7. If the user has enabled transaction notifications, a push notification appears: "₦4,500 spent at Shoprite; Food & Dining."
8. Tapping the notification opens the transaction detail view directly.

Success State Transaction appears in the app within thirty seconds of the alert arriving on the device.

Failure State Parse fails. Raw message is stored in failed_parses. No notification is sent. User does not see a broken or incomplete record in their transaction list.

7.4 Duplicate Transaction Flow

Entry Trigger The parser produces a transaction record that matches an existing record on amount, date, and merchant within a five-minute window.

Flow

1. System detects probable duplicate during pre-commit validation.
2. Duplicate is not committed to the transactions table.
3. Duplicate is logged in sync_logs with a duplicate flag and a reference to the original record ID.
4. No user notification is generated.
5. If the user later reports a missing transaction that turns out to be a suppressed duplicate, a support flow allows manual review and override.

Success State No duplicate appears in the user's transaction list. Original record is preserved intact.

Failure State A false positive duplicate detection suppresses a legitimate transaction. User notices the missing record and uses the manual entry feature or the support override flow to restore it.

7.5 Manual Transaction Entry Flow

Entry Trigger User taps the floating action button on the bottom right side on home screen or transaction list.

Flow

1. Manual entry form opens as a bottom sheet.
2. User selects transaction type: Money In or Money Out.
3. User enters amount using a numeric keypad.
4. User selects category from a scrollable list of predefined categories.
5. Date field defaults to today. User can tap to change it using a calendar picker.
6. Optional fields: merchant or payer name, description.
7. User taps Save.
8. System validates required fields. If validation passes, the record is committed.
9. Transaction appears in the list immediately with a manual entry indicator.

Success State Transaction saved and visible in the list within two seconds of tapping Save.

Failure State Required field is blank. Inline validation highlights the missing field. Form does not submit.

7.6 Export Flow

Entry Trigger User navigates to the Export screen from the main navigation.

Flow

1. Export screen displays date range picker defaulting to the current month.
2. User adjusts date range using the calendar picker.
3. Optional: user applies category or account filters using a filter panel.
4. User selects export format: PDF Report or CSV Spreadsheet.
5. User taps Generate Export.
6. Progress indicator appears while export is generated.
7. On completion, a share sheet appears with options to save to device, share via another app, or email directly.

Success State Export file is generated and delivered to the user's chosen destination within fifteen seconds for a date range of up to one year.

Failure State Export generation fails due to insufficient storage. User is notified with a specific explanation and a suggestion to free up space or narrow the date range.

7.7 Recategorization Flow

Entry Trigger User taps a transaction in the transaction list and selects Edit Category.

Flow

1. Category selection screen opens showing all predefined categories.

2. User selects the correct category.
3. System saves the new category to the transaction record.
4. If the merchant has been miscategorized before, the system surfaces a prompt: "Always categorize [Merchant Name] as [New Category]?"
5. User selects Yes, Just This One, or Cancel.
6. If Yes is selected, a new rule is added to the local rules table for that merchant.

Success State Transaction is recategorized immediately. Future transactions from the same merchant are categorized correctly if the user selected the persistent rule option.

Failure State Rule conflict detected. System presents both conflicting rules and asks the user to choose which one to keep.

7.8 Failed Email Connection Flow

Entry Trigger OAuth flow fails during email connection; either during onboarding or from the settings screen.

Flow

1. OAuth failure is detected.
2. User is returned to the email connection screen with an error message explaining the failure in plain language: "We couldn't connect to your Gmail account. This is usually a temporary issue."
3. Two options are presented: Try Again and Skip for Now.
4. If Try Again is selected, the OAuth flow is re-initiated.
5. If the failure recurs three times, the user is shown a more detailed troubleshooting message and offered a link to help documentation.

Success State OAuth completes on retry. Email ingestion begins immediately.

Failure State Repeated OAuth failure. User skips email connection. Email ingestion remains inactive. User is reminded to reconnect from settings on their next app open.

Section 8: UX Requirements

8.1 Empty States

Every screen that can appear empty must communicate value rather than absence. Empty states must never show a blank screen or a generic error illustration.

Transaction List Empty State (no permissions granted) Display an illustration of a phone receiving an alert, with the message: "Your transactions will appear here automatically once you connect your bank alerts." Include two action buttons: Connect SMS Alerts and Connect Email.

Transaction List Empty State (permissions granted, no transactions yet) Display a progress animation and the message: "SpendNG is reading your alerts. Your transactions will appear in a moment." If import takes longer than thirty seconds, add: "This may take a minute if you have a large message history."

Search Empty State (no results) Display the message: "No transactions match your search." Show which filters are currently active and include a Clear Filters button.

Export Empty State (no transactions in selected range) Display the message: "No transactions found for this date range." Include a suggestion to adjust the range.

8.2 Loading States

The product must communicate progress at every point where the user is waiting for the system.

Initial Import A dedicated import progress screen during onboarding showing the number of alerts processed so far, a progress bar, and an estimated time to completion. The user must be able to leave this screen and continue using the app while import runs in the background.

Transaction List Refresh A subtle pull-to-refresh animation. No full-screen loading state for routine refreshes.

Export Generation A progress bar with percentage completion and an estimated time remaining. The user must be able to cancel export generation.

Search Results Results must appear incrementally as they are found. No full-screen loading state for search.

8.3 Notification Behavior

Transaction Notifications Optional and off by default. When enabled, a notification is sent for every newly captured transaction showing: transaction type, amount, merchant name, and assigned category. Tapping the notification opens the transaction detail view.

Review Reminders A weekly summary notification sent every Monday morning showing: total spent last week, top spending category, and number of new transactions captured. This notification is on by default but dismissible and configurable.

Data Source Warnings If SMS ingestion has not captured a new transaction in more than seventy-two hours on a device where SMS permission is granted, the system shall send a single notification asking the user to verify that background permissions are still active. This notification is sent only once per seven-day period to avoid becoming intrusive.

8.4 Search Behavior

Search must support simultaneous filtering by amount, merchant, category, date range, account, and transaction type. All filter combinations must be valid. The search interface must make active filters visually prominent so users always understand what constraints are applied to the results they are seeing. Clearing individual filters must be possible with a single tap. Clearing all filters must be possible with a single tap.

8.5 Accessibility Requirements

Touch Targets All interactive elements must have a minimum touch target size of 40 by 40 points, regardless of the visual size of the element itself.

Typography All body text must be a minimum of 14sp (Except tooltips or captions which are 12sp). All primary labels must be a minimum of 16sp. The product must respect the user's system font size settings and must not override them.

Color Contrast All text must meet WCAG AA contrast requirements against its background. Color must never be the sole means of conveying information; category indicators must include both color and a text label or icon.

Screen Reader Support All interactive elements must include meaningful accessibility labels. Transaction list items must be announced by screen readers with: transaction type, amount, merchant name, category, and date.

Motion All animations must respect the user's system-level Reduce Motion setting. When Reduce Motion is enabled, transitions must use opacity fades rather than movement.

8.6 Error Messaging

All error messages must:

- Explain what went wrong in plain language without technical jargon.
- Explain what the user can do about it.
- Offer at least one clear action to recover.

Examples:

- Instead of "Parse error 403" write "We couldn't read this alert. You can add this transaction manually."
- Instead of "OAuth token invalid" write "Your email connection has expired. Tap here to reconnect."
- Instead of "Insufficient storage" write "Your device doesn't have enough space to generate this export. Free up some storage and try again."

8.7 Offline Behavior

SpendNG must be fully functional without an internet connection for all local features. SMS ingestion, transaction parsing, categorization, search, manual entry, and the home dashboard must all function offline. Export to device must function offline. Email ingestion and cloud backup require connectivity and must communicate their inactive status gracefully when offline rather than failing silently.

Section 9: Data Model

9.1 users

Purpose Stores account information for each SpendNG user.

Column	Type	Notes
user_id	UUID	Primary key, system generated
email	VARCHAR(255)	Unique, required
phone_number	VARCHAR(20)	Optional, unique where provided
password_hash	VARCHAR(255)	Bcrypt hashed
display_name	VARCHAR(100)	Optional
created_at	TIMESTAMP	Required
last_active_at	TIMESTAMP	Updated on each app open
account_status	ENUM	active, suspended, deleted

Relationships One user has many transactions, many sync_logs, and many rules.

Indexes Unique index on email. Unique index on phone_number where not null.

9.2 transactions

Purpose Stores every financial record captured or manually entered by the user.

Column	Type	Notes
transaction_id	UUID	Primary key, system generated
user_id	UUID	Foreign key → users
amount	DECIMAL(15,2)	Required, always positive
currency	CHAR(3)	ISO 4217, default NGN
transaction_type	ENUM	credit, debit
merchant_name	VARCHAR(255)	Extracted or user entered
category_id	INTEGER	Foreign key → categories
account_name	VARCHAR(100)	Bank or fintech platform name
account_number_last4	CHAR(4)	Last four digits where available
transaction_date	TIMESTAMP	Date and time of transaction
available_balance	DECIMAL(15,2)	Optional, from alert where present
source_channel	ENUM	sms, email, manual
raw_source_text	TEXT	Original alert text, stored for audit
parse_confidence	ENUM	high, medium, low, manual
is_duplicate	BOOLEAN	Default false
is_flagged_for_review	BOOLEAN	Default false
notes	TEXT	Optional user notes

created_at	TIMESTAMP	Record creation time
updated_at	TIMESTAMP	Last modification time

Relationships Many transactions belong to one user. Many transactions belong to one category.

Indexes Index on user_id. Index on transaction_date. Index on category_id. Composite index on user_id and transaction_date for dashboard queries. Full-text index on merchant_name and notes for search.

9.3 categories

Purpose Stores the predefined financial categories used for transaction classification.

Column	Type	Notes
category_id	INTEGER	Primary key, auto-increment
name	VARCHAR(100)	Required, unique
icon_identifier	VARCHAR(50)	Reference to icon asset
color_hex	CHAR(7)	Display color for UI
transaction_type_hint	ENUM	credit, debit, both
is_system	BOOLEAN	True for predefined, false for user-created
created_at	TIMESTAMP	

Relationships One category has many transactions.

Indexes Index on name. Index on is_system.

9.4 rules

Purpose Stores keyword and pattern matching rules used by the categorization engine. Includes both system-defined rules and user-created rules derived from manual corrections.

Column	Type	Notes

rule_id	UUID	Primary key
user_id	UUID	Foreign key → users, null for system rules
keyword	VARCHAR(255)	Merchant name or keyword pattern
match_type	ENUM	exact, contains, starts_with, regex
category_id	INTEGER	Foreign key → categories
priority	INTEGER	Higher number = higher priority
is_system	BOOLEAN	True for default rules
created_at	TIMESTAMP	
last_applied_at	TIMESTAMP	Last time this rule matched a transaction

Relationships Many rules belong to one user. Many rules reference one category.

Indexes Index on user_id. Index on keyword. Composite index on user_id and priority for rule resolution order.

9.5 sync_logs

Purpose Records every ingestion event; successful parse, failed parse, duplicate detection, or OAuth refresh; for debugging, support, and analytics.

Column	Type	Notes
log_id	UUID	Primary key
user_id	UUID	Foreign key → users
source_channel	ENUM	sms, email
sender_identifier	VARCHAR(100)	Sender ID or email address
raw_content_hash	CHAR(64)	SHA-256 of raw message, for deduplication
parse_status	ENUM	success, failed, duplicate, skipped
transaction_id	UUID	Foreign key → transactions, null if parse failed

failure_reason	TEXT	Populated on failed or skipped status
processed_at	TIMESTAMP	

Relationships Many sync_logs belong to one user. Successful sync_logs reference one transaction.

Indexes Index on user_id. Index on processed_at. Index on parse_status. Index on raw_content_hash for duplicate detection.

9.6 failed_parses

Purpose Stores raw messages that could not be parsed successfully, enabling manual review, pattern improvement, and user-initiated recovery.

Column	Type	Notes
failed_parse_id	UUID	Primary key
user_id	UUID	Foreign key → users
source_channel	ENUM	sms, email
sender_identifier	VARCHAR(100)	
raw_content	TEXT	Original message content
failure_reason	TEXT	Why parsing failed
is_reviewed	BOOLEAN	Default false
reviewed_at	TIMESTAMP	
created_at	TIMESTAMP	

Indexes Index on user_id. Index on is_reviewed. Index on created_at.

Section 10: Technical Architecture

10.1 Guiding Principles

SpendNG's technical architecture is shaped by three constraints that are non-negotiable given the product's context and user base.

Privacy by default. Transaction data is among the most sensitive personal data that exists. The architecture must minimize the amount of data that leaves the user's device. Parsing, categorization, and search must happen on-device wherever technically feasible. Data that is stored in the cloud must be encrypted at rest and in transit.

Resilience on low-end hardware. A significant portion of the target user base operates on Android devices with limited RAM, aggressive battery management, and intermittent connectivity. The architecture must degrade gracefully under these conditions rather than failing entirely.

Maintainability without a large team. SpendNG version 1 will be built and maintained by a small team. Every architectural decision must favor simplicity and debuggability over technical sophistication.

10.2 Frontend

Framework React Native, targeting Android and iOS from a single codebase. React Native is selected over Flutter because the JavaScript ecosystem offers stronger libraries for SMS handling, OAuth integration, and PDF generation on both platforms.

State Management Zustand for global application state. Lightweight, minimal boilerplate, and sufficient for the data complexity of version 1.

Local Database WatermelonDB for on-device transaction storage. WatermelonDB is built for React Native, supports lazy loading for large datasets, and provides the performance characteristics needed for search across large transaction histories on mid-range Android devices.

Navigation React Navigation with a bottom tab navigator for primary navigation and a stack navigator for modal flows including transaction detail, export, and settings.

UI Component Library Custom component library built on React Native core components, using a consistent design token system for color, typography, and spacing. No third-party UI kit to avoid bundle size overhead and maintain design control.

10.3 Backend

Platform Supabase, providing PostgreSQL database, authentication, storage, and edge functions from a single managed platform. Supabase is selected for its developer velocity,

generous free tier for early-stage products, and built-in row-level security which is essential for multi-user transaction data.

Authentication Supabase Auth with email and password sign-in. Google OAuth for social sign-in. JWT tokens managed by Supabase with automatic refresh.

Database Supabase PostgreSQL hosting the users, transactions, categories, rules, sync_logs, and failed_parses tables as defined in the data model. Row-level security policies ensure users can only access their own data.

Cloud Functions Supabase Edge Functions (Deno runtime) for server-side operations including: export PDF generation for large date ranges that exceed on-device memory capacity, and background email polling for connected Gmail and Outlook accounts.

File Storage Supabase Storage for generated export files awaiting download. Files are stored with user-scoped access policies and deleted after seven days.

10.4 On-Device Services

SMS Reader Service (Android) A React Native background service using the react-native-sms-retriever library for real-time SMS monitoring and a one-time inbox scan on installation. The service is registered as a foreground service on Android to reduce the likelihood of being killed by aggressive battery management. A persistent notification (low priority, collapsible) is displayed as required by Android's foreground service policy, with copy that communicates value: "SpendNG is watching for new transactions."

Email Polling Service A background job scheduled via react-native-background-fetch that polls the connected Gmail or Outlook account every fifteen minutes when the device has connectivity. The polling interval is configurable in settings between five minutes and one hour.

On-Device Parser A JavaScript parsing engine that runs entirely on-device. Parser templates for each supported bank and fintech sender are bundled with the app and updated via over-the-air updates using Expo EAS Update. This means new bank support can be added without a full app store release.

On-Device Categorization Engine A rule-matching engine that applies the categories and rules tables stored in WatermelonDB. System rules are bundled with the app. User rules are stored locally and synced to Supabase for backup.

Local Search Index WatermelonDB's built-in query engine handles search for most use cases. For full-text search across merchant names and notes, a lightweight on-device FTS index is maintained using SQLite's FTS5 extension, which WatermelonDB exposes.

10.5 Security Architecture

Data Encryption at Rest WatermelonDB's local database is encrypted using SQLCipher with a key derived from the user's device PIN or biometric authentication token. Transaction data on device is unreadable without the user's authentication.

Data Encryption in Transit All communication between the app and Supabase uses TLS 1.3. No transaction data is transmitted over unencrypted connections under any circumstances.

OAuth Token Storage OAuth tokens for Gmail and Outlook are stored in the device's secure keychain (iOS Keychain, Android Keystore). They are never stored in plain text or in WatermelonDB.

SMS Data Handling Raw SMS content is processed on-device and never transmitted to any server in its raw form. Only structured transaction records are synced to Supabase. The raw_source_text field stored in the transactions table contains the original alert text for audit purposes but is encrypted at the database level.

Row-Level Security All Supabase tables have row-level security policies enforced at the database level, ensuring that a compromised application token cannot access another user's data.

Data Minimization SpendNG collects only what is necessary to deliver its core functionality. No advertising identifiers, behavioral analytics SDKs, or third-party tracking libraries are included in the application.

10.6 Over-the-Air Updates

Expo EAS Update is used for over-the-air JavaScript bundle updates. This allows parser templates, categorization rules, and UI fixes to be deployed without requiring a full App Store or Play Store release cycle. OTA updates are limited to the JavaScript layer and cannot modify native modules, which must go through the standard store review process.

10.7 Security Checklist Before Every Push to Github or Launch

Before SpendNG goes live, you should confirm every one of these, when available:

- WatermelonDB is encrypted with SQLCipher and the key is derived from user authentication
- OAuth tokens are stored exclusively in iOS Keychain and Android Keystore via react-native-keychain
- All Supabase API keys are in environment variables and excluded from version control
- Row-Level Security policies are written and tested for every table and every operation
- The app makes no HTTP requests, only HTTPS
- Raw SMS content is never transmitted to any server
- A privacy policy is written, reviewed, and linked from the App Store listing

- A data deletion flow exists so users can request full account and data removal
- Supabase database backups are enabled and you have tested restoration
- You have a written plan for what you will do in the first 24 hours if a breach is discovered

Section 11: Analytics and Success Metrics

11.1 Principles

SpendNG measures success through behavioral signals that indicate genuine utility, not vanity metrics. Install counts and registration numbers are not success metrics. The product succeeds when users return to it regularly because it is giving them something they could not easily get elsewhere.

All analytics are collected using a self-hosted Plausible instance to avoid sending user behavioral data to third-party advertising platforms. No user-identifiable information is included in any analytics event. Events are tied to anonymous session identifiers only.

11.2 Primary Success Metrics

Day-7 Retention Rate The percentage of users who open SpendNG at least once in the seven days following their first session. Target: 40% at end of public beta. Rationale: a user who returns after seven days has seen the product work passively for a week. Their return indicates the automatic capture is delivering perceived value.

Day-30 Retention Rate The percentage of users who open SpendNG at least once in the thirty days following their first session. Target: 25% at end of public beta. A user retained at thirty days is likely to become a long-term user.

Parsing Accuracy Rate The percentage of SMS and email alerts that are successfully parsed into complete transaction records with high or medium confidence. Target: 90% or above across supported senders. Measured by dividing successful parses by total ingestion attempts in sync_logs.

Categorization Accuracy Rate The percentage of automatically categorized transactions that are not subsequently recategorized by the user. Target: 80% or above. Measured by comparing total transactions to total user-initiated recategorizations.

Export Completion Rate The percentage of users who initiate an export and successfully complete it. Target: 85% or above. A low completion rate indicates friction in the export flow that must be resolved, given that export is a primary value moment for Amara and Chidi.

11.3 Secondary Metrics

Transactions Captured Per User Per Month Average number of transactions automatically captured per active user per month. This metric indicates whether ingestion is working at the volume needed to make summaries and search meaningful.

Manual Entry Rate Percentage of total transactions that are manually entered versus automatically captured. A high manual entry rate suggests that automatic ingestion is failing for a significant user segment, which warrants investigation.

Permission Grant Rate Percentage of users who grant SMS permission during onboarding. Percentage who connect email. These rates directly measure the effectiveness of the permission explanation screens.

Failed Parse Rate by Sender Breakdown of failed parses by bank or fintech sender. This identifies which institutions have alert formats not yet covered by the parser and prioritizes template development.

Monthly Active User Rate Percentage of registered users who open the app at least once per month. Target: 35% at end of public beta.

Search Usage Rate Percentage of monthly active users who use the search feature at least once per month. This indicates whether users are finding value beyond passive capture.

11.4 Anti-Metrics

These are metrics SpendNG explicitly does not optimize for, because optimizing for them would compromise the product's integrity.

Session Length. SpendNG is not a social or entertainment product. Long sessions may indicate confusion rather than engagement. The product succeeds when users get what they need quickly and leave.

Notification Volume. More notifications do not mean more engagement. SpendNG should earn attention, not demand it. Notification open rate is a health metric, not a growth lever.

Feature Discovery Rate. Forcing users into features they did not seek out creates friction and resentment. Feature adoption should be organic and driven by user need.

Section 12: Launch and Validation Plan

12.1 Phase 1; Solo Dogfooding

Duration Two to four weeks following internal build completion.

Who The founding team only; ideally the primary product owner running SpendNG as their sole financial tracking tool across all their personal accounts.

Goals

- Verify that SMS ingestion works reliably across the specific banks and fintech apps used daily.
- Identify parser failures, miscategorizations, and UX friction through personal daily use.
- Confirm that background service survival is acceptable across the primary test device.
- Produce at least one full month of transaction data and validate that the export output is accurate and presentable.

Risks

- Test environment too narrow; the founding team may use fewer banks than the broader user base, leaving coverage gaps undiscovered.
- Personal familiarity with the product creates blind spots in UX evaluation.

Success Criteria

- Parsing accuracy above 85% across all senders in personal use.
- No critical crashes over a seven-day continuous use period.
- Export produces a PDF and CSV that are accurate to within 2% of manually verified transaction totals.
- The founding team would genuinely recommend the product to someone they care about.

Rollback Conditions If parsing accuracy falls below 70% or the background service fails to survive device restarts on primary test hardware, phase 2 is delayed until the issues are resolved.

12.2 Phase 2; Friends and Family Beta

Duration Four to six weeks following successful phase 1 completion.

Who Twenty to fifty users recruited from the founding team's personal network, selected to represent all three personas; salary earners, freelancers, and small business owners; and covering a diverse range of devices and banks.

Goals

- Validate parsing accuracy across a wider range of SMS alert formats and bank senders than the founding team uses personally.
- Identify onboarding failure points; where do real users drop off before completing setup.
- Collect qualitative feedback on categorization quality, search usability, and export output.
- Measure day-7 and day-14 retention for the first time.
- Identify the most common banks and fintechs not yet supported by the parser.

Distribution TestFlight for iOS. Internal track on Google Play for Android. Direct invite links only.

Feedback Collection An in-app feedback button accessible from settings that submits a structured report with device information, app version, and a free-text description. A private WhatsApp group for real-time feedback from beta participants. Weekly structured check-in questions sent via the WhatsApp group.

Risks

- Friends and family may be too polite to report genuine problems.
- Device diversity on Android may surface background service failures not seen on founding team devices.
- Some participants may lose interest after the first week if their bank is not yet supported.

Success Criteria

- Parsing accuracy above 88% across all senders encountered in beta.
- Day-7 retention above 35%.
- Onboarding completion rate above 70% (user reaches home screen with at least one transaction visible).
- At least fifteen of twenty users report that the product has captured their transaction history accurately enough to be useful.
- Export feature used by at least 60% of beta participants at least once.

Rollback Conditions If day-7 retention falls below 20%, the product is paused for a fundamental review of the value proposition delivery; specifically whether passive capture is working reliably enough to generate perceived value. If a critical privacy issue is identified; any scenario where non-transaction SMS content could be exposed; phase 3 is suspended immediately until the issue is resolved and independently reviewed.

12.3 Phase 3; Public Beta

Duration Eight to twelve weeks following successful phase 2 completion.

Who Open to the public via App Store and Play Store listings, with a waitlist mechanism to control growth rate and allow infrastructure to scale without surprise.

Goals

- Validate product-market fit across a demographically and geographically diverse Nigerian user base.
- Stress-test backend infrastructure and on-device performance at scale.
- Achieve day-30 retention target of 25% or above.
- Identify the top five feature requests that should inform the version 2 roadmap.
- Generate the first real revenue signal if a freemium or subscription model is introduced at this stage.

Distribution Public App Store and Play Store listings. Organic social media presence. Partnerships with Nigerian personal finance communities and Twitter finance spaces.

Support A dedicated support email address and an in-app help center covering the most common issues identified in phase 2. A public changelog so users understand what is being improved.

Risks

- Unexpected device or carrier configurations that break SMS ingestion at scale.
- Negative App Store reviews from users whose banks are not yet supported, creating a perception problem before parser coverage is complete.
- Rapid user growth overwhelming backend infrastructure before it is optimized for scale.
- A privacy concern amplified on social media creating reputational damage.

Success Criteria

- Ten thousand registered users by end of phase 3.
- Day-30 retention above 25%.
- Parsing accuracy maintained above 88% as new bank formats are encountered and added.
- App Store rating of 4.2 or above.
- At least one unprompted positive public mention from a user in a Nigerian personal finance community.
- Infrastructure cost per active user at a level sustainable for a bootstrapped or lightly funded operation.

Rollback Conditions If a privacy vulnerability is discovered that exposes user transaction data, the app is removed from both stores immediately, affected users are notified directly, and the product does not return to public availability until an independent security review is completed. If day-30 retention falls below 15% across a cohort of at least five hundred users, the product is withdrawn for a fundamental review before further public investment.